Método de ...

Javier Alexander López Contreras
Facultad de Ciencias Físico Matemáticas (FCFM), Universidad Autónoma de Coahuila, Campus Campo redondo

Recibido el 20 de Marzo del 2026

En este documento se analizará el código en Phyton del método de Müller; que es un método donde se utilizan 3 valores para encontrar la aproximación de la raíz de un polinomio. Se busca explicar de mejor manera el código.

1. INTRODUCCIÓN

El estudio de los métodos numéricos es esencial para hallar soluciones a ecuaciones donde los métodos analíticos resultan insuficientes. Mientras que el método de la secante utiliza una aproximación lineal basada en dos puntos, el Método de Müller es un algoritmo de búsqueda de raíces que destaca por su capacidad de encontrar tanto raíces reales como complejas. A diferencia de los métodos de bisección o secante, Müller utiliza una aproximación cuadrática, lo que generalmente le otorga una convergencia más rápida y una mayor versatilidad ante funciones polinomiales y trascendentes.

2. ALGORITMO Y EXPLICACIÓN DEL MÉTODO DE MÜLLER

El método de Müller se basa en proyectar una parabola a travez de 3 puntos iniciales, x_0, x_1 y x_2. El proceso lógico sigue estos pasos:

- **Ajusto cuadrático**: Se construye un polinómio de grado $P(x) = a(x - x_2)^2 - b(x - x_2) + c$ que pasa por los 3 puntos.
- **Cálculo de coeficientes**: Se determinan los coeficientes a,b y c basándose en las diferencias de la función en los puntos evaluados.
- **Intersección con el eje x**: La nueva aproximación x_3 se consigue encontrando la raíz del polinomio cuadrático más cercana a x_2. Para evitar errores de redondeo, se utiliza la variante de la fórmula cuadrática:

$$x_3 = x_2 - \frac{2c}{b \pm \sqrt{b^2 - 4ac}} \tag{1}$$

- **Criterio de selección**: Se elige el signo del denominador que maximice su magnitud, haciendo así un paso más estable hacia la raíz.

3. ANÁLISIS DEL CÓDIGO

Para validar el método, se desarrolló un programa que traduce la lógica matemática a instrucciones de control. Los puntos clave de la implementación son:

- **Cálculo de parametros y coeficientes**: El algoritmo de Müller requiere determinar una parábola que pase por tres puntos. En el código, esto se resuelve calculando primero las distancias (h_1, h_2) y las diferencias de la función (e_0, e_1) respecto al tercer punto (x_2):

```
    c = f(x2)
    h1 = x1 - x2
    h2 = x0 - x2
    e0 = f(x0) - c
    e1 = f(x1) - c

```

Listing 1: Definición de variables auxiliares para la construcción del polinomio cuadrático.

- **Resolución del Sistema para a y b**: Se define una variable w para manejar el denominador del sistema de ecuaciones. El código incluye un control preventivo: si w es cero, significa que la pendiente es nula o los puntos no permiten formar una parábola.

```
    w = (h1 * (h2**2)) - (h2 * (h1**2))
if w == 0:
    resultado.config(text='Error: w es cero (
     divisi n imposible)', fg='red')
    break
a = (e0 * h1 - e1 * h2) / w
b = (e1 * (h2**2) - e0 * (h1**2)) / w

```

Listing 2: Cálculo de los coeficientes de la parábola y manejo de indeterminaciones.

- **Manejo de Números Complejos**: A diferencia de otros métodos abiertos, Müller puede hallar raíces en el plano complejo. El código fuerza este comportamiento sumando 0j al discriminante.

```
    discriminante = np.sqrt(b**2 - 4*a*c + 0j
    )

```

Listing 3: Implementación del discriminante con soporte para números imaginarios.

- **Criterio de Estabilidad en el Denominador**: Para asegurar que el algoritmo avance hacia la raíz de forma estable, se elige el signo que maximice el valor absoluto del denominador.

```
    den_pos = b + discriminante
    den_neg = b - discriminante
    denominador = den_pos if abs(den_pos) >
    abs(den_neg) else den_neg

```

Listing 4: Optimización del denominador para evitar errores de redondeo y maximizar la convergencia.

- **Actualización de Estados**: Para que el algoritmo avance, los valores de los puntos deben desplazarse en cada ciclo iterativo.

```
    x0, x1, x2 = x1, x2, x3
    iter_count += 1

```

Listing 5: Desplazamiento de puntos para la siguiente iteración.

4. RESULTADOS Y ANÁLISIS

Se seleccionaron tres escenarios críticos para evaluar el desempeño de la implementación del Método de Müller.

4.1. Raíces reales de polinomios.

Utiliza la función que tienes por defecto: $f(x) = x^3 - 13x - 12$.

- **Configuración**: $x_0 = 4.5, x_1 = 4.8, x_2 = 5$.
- **Análisis**: Esta prueba valida la convergencia cuadrática del método en polinomios de grado superior. Müller debería encontrar la raíz exacta ($x = 4$) en muy pocas iteraciones, demostrando su eficiencia sobre métodos lineales.

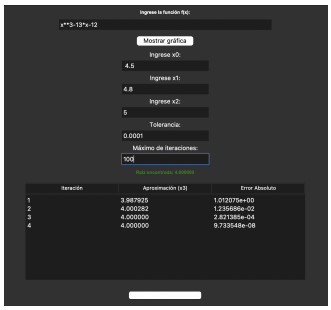

Figura 1: Convergencia del método para una raíz real en un polinomio de tercer grado.

4.2. Raíces Complejas

Utiliza la función: $f(x) = x^2 + x + 1$.

- **Configuración**: $x_0 = -1, x_1 = 0, x_2 = 1$.
- **Análisis**: A diferencia del método de la secante o bisección, Müller puede identificar raíces imaginarias gracias al manejo del discriminante complejo en el código (0j). El programa debería arrojar un resultado cercano a $-0.5 \pm 0.8660j$.

Figura 2: Identificación de raíces complejas mediante el uso de aritmética de números imaginarios en el discriminante.

4.3. Funciones Trascendentes y Estabilidad

Utiliza la función: $f(x) = e^{-x} - x$

- **Configuración**: $x_0 = 0, x_1 = 0.5, x_2 = 1$
- **Análisis**: Se evalúa la capacidad del método para manejar funciones no polinomiales. Es ideal para comparar la velocidad de convergencia contra el método de bisección, aprovechando que Müller utiliza la curvatura de la función para aproximarse más rápido al cero.

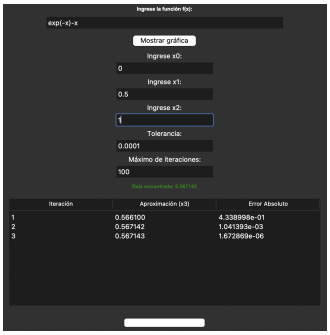

Figura 3: Comportamiento del método ante una función exponencial trascendente.

5. CONCLUSIONES

El análisis permite concluir que el método de Müller es una de las herramientas más potentes en el análisis numérico debido a su rapidez y su capacidad de detectar raíces múltiples o complejas. Su implementación computacional demuestra que, aunque es matemáticamente más complejo que la secante, su robustez compensa el costo de cálculo adicional, siempre que se seleccionen adecuadamente los puntos iniciales.

6. Bibliografía

- Burden, R. L., Faires, J. D. (2011). Análisis Numérico. 9na Edición. Cengage Learning.
- Chapra, S. C., Canale, R. P. (2011). Métodos Numéricos para Ingenieros. 6ta Edición. McGraw-Hill.
- Mathews, J. H., Fink, K. K. (2000). Métodos Numéricos con MATLAB. 3ra Edición. Prentice Hall.
- Van Rossum, G., Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace.
- Harris, C. R., Millman, K. J., et al. (2020). Array programming with NumPy. Nature, 585(7825), 357-362.